SOFTWARE ENGINEERING
FINAL PROJECT REPORT

PROJECT NAME : THE BREW-TIME (SMART ORDERING SYSTEM)

TEAM MEMBERS : AbdElrahman Elsayed Atta (2305153)
Zyad Ehab Mohamed (2305147)
Faten yasser (2305171)
COURSE : SOFWARE ENGINEERING

TABLE OF CONTENTS

1. COVER PAGE
2.SOFTWARE DEVELOPMENT LIFECYCLE (SDLC)
   i. Requirments Phase
   ii. Design Phase
   iii. Implementation Phase
   iv.Testing Phase
   v. Maintenance Phase.

Software Development Lifecycle (SDLC)

2.1 Requirments Phase:

2.1.1 problem statment

Alexandria National University's campus coffee shops suffer from persistent operational inefficiencies: long queues during peak hours, manual order-taking that leads to frequent miscommunication, and a complete lack of real−time feedback for waiting customers. These issues result in customer dissatisfaction, wasted time, and uneven workload for the baristas. "The Brew−Time" addresses this by providing a digital self−service ordering platform where customers can browse the menu, place orders instantly, and track their order status. Staff receive a live dashboard to manage incoming orders and update their progress, eliminating paper slips and verbal mix−ups.

2.1.1 System scope:

The Brew−Time is a web−based application designed specifically for the on−campus coffee shop at the Faculty of Computer and Information. The system will be accessible via modern web browsers on both desktop and mobile devices.

In scope:

1. Displaying a digital menu with product names, descriptions, and prices.
2. Allowing customers to add items to a cart and submit an order.
3. Generating a unique Order ID and storing the order in a persistent queue.
4. Providing a staff (barista) dashboard that lists all active orders in chronological order.
5. Allowing staff to update an order's status through predefined stages: Pending → Preparing → Ready.
6. Real−time (or near−real−time) update of order status visible to the customer.

Out of Scope:

1. Payment processing (the system assumes cash or external payment; no financial transactions are handled).
2. User account registration or login (orders are placed as a guest for simplicity; staff access may use a static password).
3. Inventory management or stock tracking.
4. Analytics dashboards for sales reports.
5. Multi−shop or chain support.

Actors:

1. Customer: Browses the menu, places an order, and views the real−time status of their own order.
2. Barista (Staff): Views the queue of pending orders, updates order statuses, and marks orders as completed.

2.1.3 Functionl Requirments

The following detailed functional requirements specify what the system must do. Each requirement is uniquely identified.

ID	Requirement Description
FR1	The system shall display a digital menu with product name, description, and price for all availab
FR2	Customers shall be able to select a product and add it to a shopping cart.
FR3	Customers shall be able to adjust item quantities or remove items from the cart before checkou
FR4	The system shall allow the customer to submit their cart as a final order by clicking a “Place Or

ID	Requirement Description
FR5	Upon successful order placement, the system shall generate a unique Order ID (e.g., sequentia and display it to the customer.
FR6	The system shall store every order with the following data: Order ID, list of items, total price, ti and initial status “Pending”.
FR7	Staff shall be able to view a dashboard listing all active orders (status ≠ Completed) sorted by t showing Order ID, items, and current status.
FR8	Staff shall be able to change the status of an order. The allowed transitions are: Pending → Prep Ready → Completed.
FR9	The system shall refresh the staff dashboard automatically (or via manual refresh) to reflect ne orders within 2 seconds.
FR10	Customers shall be able to enter their Order ID on a dedicated “Track Order” page to see the cu status.
FR11	The system shall display a visually distinct status for each order stage on the customer tracking coloured progress bar).
FR12	The system shall prevent accidental duplicate order submissions if the customer refreshes the confirmation page.
FR13	The system shall validate that the cart is not empty before allowing order placement; an appro message shall be displayed if empty.
FR14	Staff must be able to mark an order as “Completed”, which removes it from the active dashboa

2.1.4 Non Functional Requirments

The following quality attributes are categorised as required by the project guidelines.

Performance:

NFR1: The staff dashboard shall display a newly placed order within 2 seconds of the customer submitting it.
NFR2: All page loads (menu, tracking) shall complete within 3 seconds on a standard university Wi−Fi connection.

Security:
NFR3: All user inputs (product IDs, quantities, Order IDs) shall be sanitised to prevent SQL injection and cross−site scripting (XSS) attacks.
NFR4: The staff dashboard shall be protected by a simple authentication mechanism (e.g., a hard−coded access code) to prevent unauthorised status changes.

Usability:
NFR5: The customer shall be able to complete an order from menu browse to confirmation in no more than 4 clicks/taps.
NFR6: The user interface shall be responsive and legible on mobile screens (at least 320px width), as most campus users will access the system via smartphones.
NFR7: Order status terms (“Preparing”, “Ready”) shall be displayed in Arabic and English to accommodate all staff and students.

Reliability:
NFR8: The system shall maintain the order queue intact even if the server is restarted or the browser is refreshed (data persists in a database).
NFR9: In case of a lost network connection, the customer shall receive a clear error message and be able to resubmit the order without data corruption.

Scalability:
NFR10: The database schema shall be designed to support additional product categories and customisation options (e.g., size, milk type) without requiring structural changes.

NFR11: The system shall be deployable on a lightweight server (e.g., a Raspberry Pi or low−end cloud instance) while supporting up to 200 concurrent orders per day.

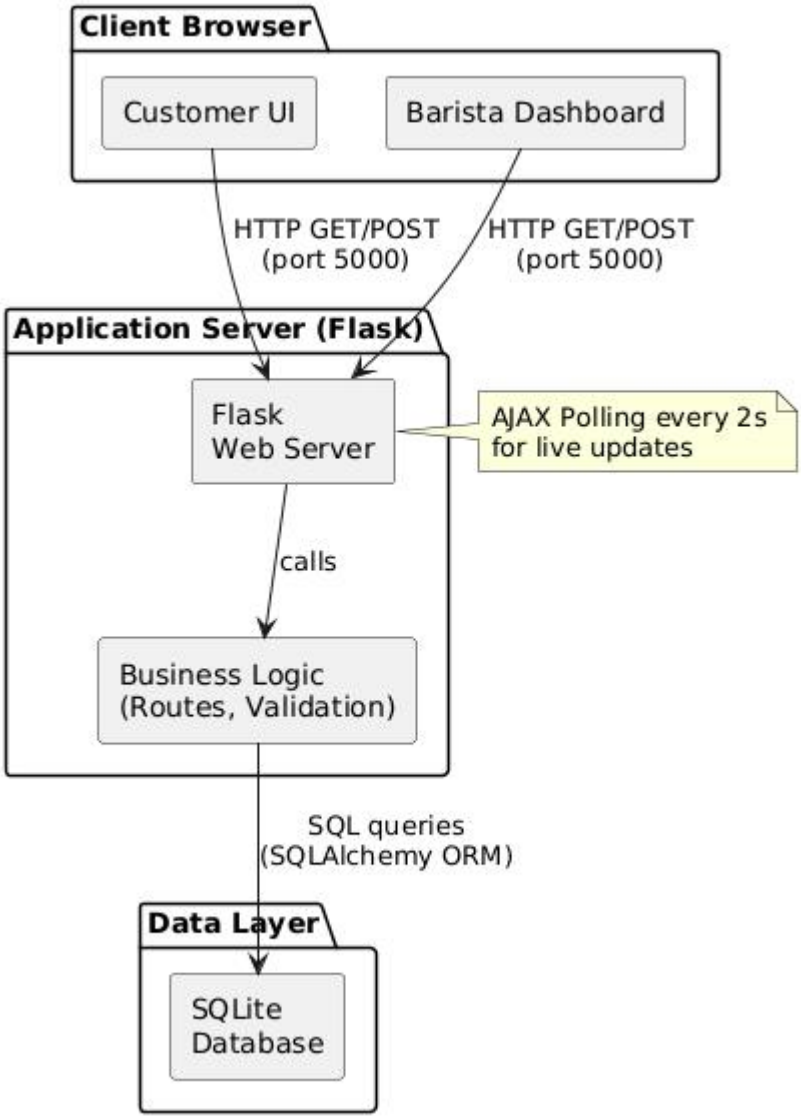
2.2 Design Phase

2.2.1 System Architucture

The Brew-Time follows a client−server architecture implemented as a monolithic web application. The system is divided into three logical tiers:

1. Presentation Tier (Frontend): HTML5, CSS3 (Bootstrap), and vanilla JavaScript running in the user's browser. The customer interface and barista dashboard are separate pages within the same application.
2. Application Tier (Backend): Python with the Flask micro−framework, handling all business logic, routing, and API responses. The server renders templates and serves static assets.
3. Data Tier: SQLite, a serverless relational database, accessed via Flask−SQLAlchemy ORM. The database contains tables for products, orders, and order_items.

Communication between the tiers occurs over HTTP. For real−time updates on the barista dashboard and the customer tracking page, the frontend uses periodic AJAX polling (every 2 seconds) to fetch the latest order statuses from RESTful endpoints. This design was chosen for simplicity and ease of deployment on a lightweight campus server. A high−level architectural diagram is provided below.


Client Browser
Customer UI
Barista Dashboard
HTTP GET/POST
(port 5000)
HTTP GET/POST
(port 5000)
Application Server (Flask)
Flask
Web Server
AJAX Polling every 2s
for live updates
calls
Business Logic
(Routes, Validation)
SQL queries
(SQLAlchemy ORM)
Data Layer
SQLite
Database

2.2.2 Design Decisions

The following key decisions were made to balance the project requirements with practical feasibility:

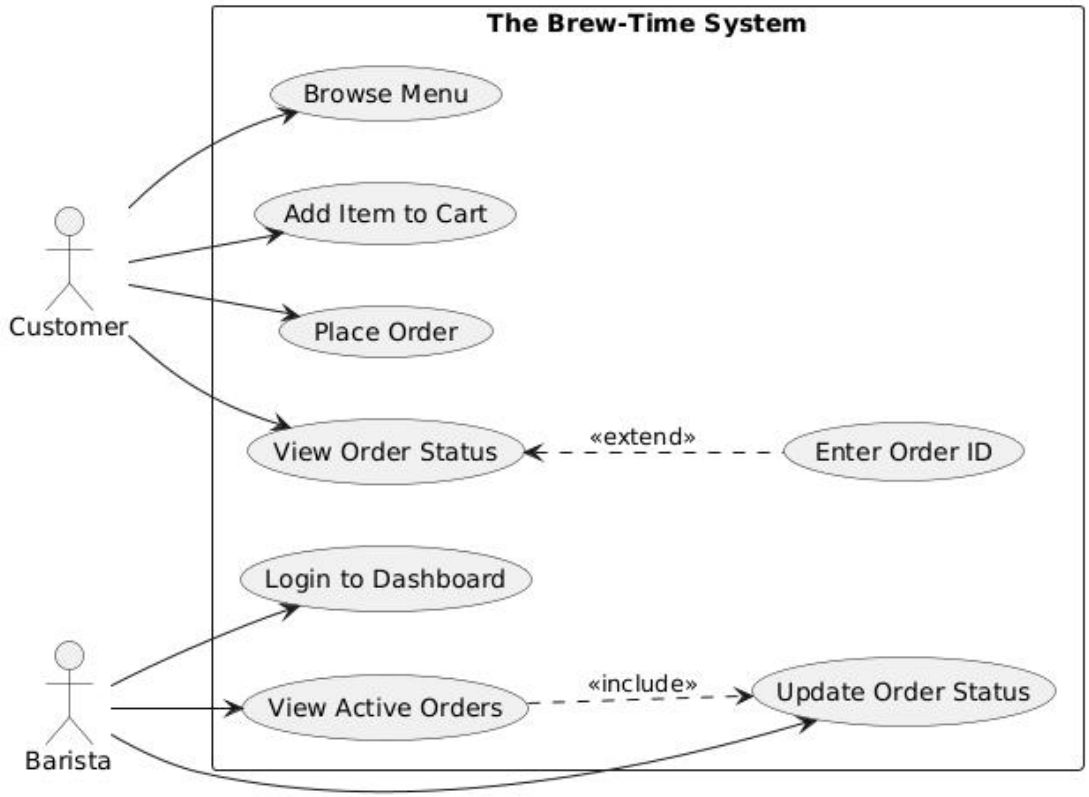
1. Flask + SQLite over heavier stacks: Flask is minimal and easy to learn, allowing the team to focus on software engineering practices rather than framework complexity. SQLite eliminates the need for a separate database server, which is ideal for a small−scale campus application that will handle up to 200 orders per day.
2. AJAX Polling instead of WebSockets: Real−time bidirectional communication (WebSockets) would be technically superior, but AJAX polling is far simpler to implement and test. A 2−second refresh interval satisfies the non−functional requirement of timely dashboard updates without adding infrastructure complexity.
3. Guest ordering without user accounts: The system targets a fast in−and−out environment. Forcing registration would violate the usability goal of under four clicks. A unique Order ID is used for tracking, and the barista dashboard is protected by a simple access code rather than a full authentication system.
4. Monolithic structure over microservices: The entire application is a single codebase, making it easy to develop, debug, and deploy as a final project. The code is separated into clear modules (models.py, routes.py, templates) to mimic a clean architecture.
5. RESTful API design for status updates: The barista's status change actions are exposed as REST endpoints (POST /update_status/<order_id>). This cleanly separates the frontend from the backend and allows the team to write clear test cases.

2.2.4 UML Diagrams

1. Use Case Diagram

Purpose: To model the interactions between actors (Customer, Barista) and the system, showing the high−level functionality from the user's perspective.

The diagram contains:
1. Actors: Customer and Barista.
2. System boundary: "The Brew-Time System".
3. Use cases (Customer): Browse Menu, Add Item to Cart, Place Order, View Order Status.
4. Use cases (Barista): Login to Dashboard, View Active Orders, Update Order Status.
5. Relationships: Barista has a Login to Dashboard use case, which is a precondition for the other barista use cases. View Order Status for the customer is extended by Enter Order ID.

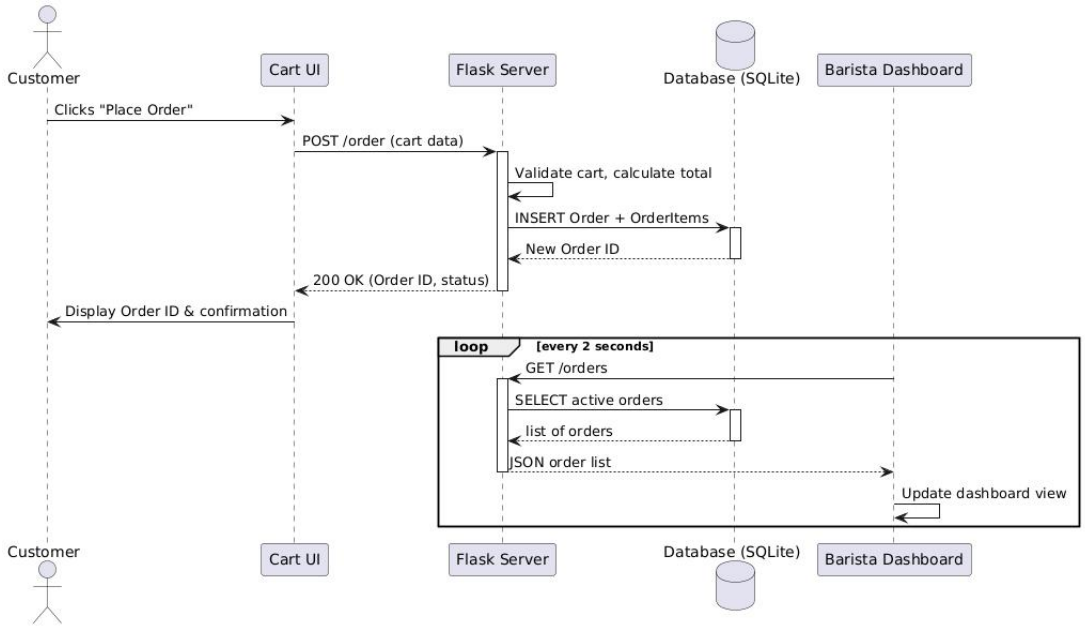


2 sequence diagram

Purpose: To detail the chronological flow of messages between objects for the critical “Place Order” scenario.

The diagram includes the following lifelines:
1. Customer (actor)
2. Cart UI (frontend page)
3. Flask Server (backend)
4. Database (SQLite)
5. Barista Dashboard (separate UI)

The flow:
1. Customer browses menu and adds items (not shown in detail to keep the diagram focused).
2. Customer clicks “Place Order”.
3. Cart UI sends POST /order with cart data to Flask Server.
4. Flask Server validates the cart, calculates total, and sends an INSERT query to the Database.
5. Database returns the new Order ID.
6. Flask Server responds to Cart UI with the Order ID and a success message.
7. Cart UI displays the Order ID to the Customer.
8. Meanwhile, the Barista Dashboard (using a timer) continuously sends GET /orders requests. After the new order is saved, the dashboard will fetch it and display it to the Barista.

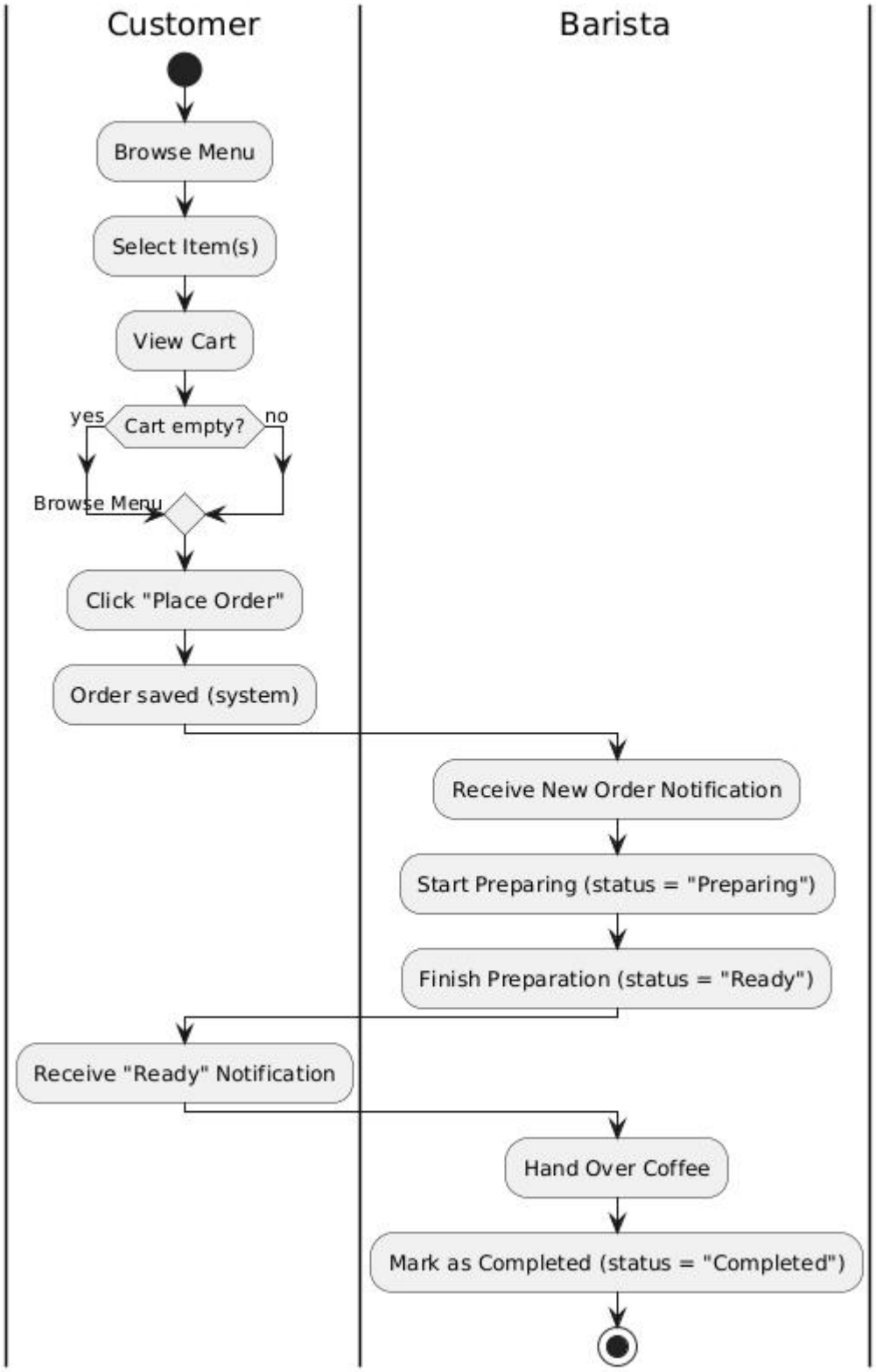


3 Activity Diagram

Purpose: To model the entire order processing workflow from the customer’s action to barista completion.

he diagram contains one swimlane for Customer and one for Barista. The flow is:

1. Start: Customer opens app.
2. Customer: Browse Menu → Select Item(s) → View Cart.
3. Decision: Cart empty? → If yes, return to Browse; if no, proceed.
4. Customer: Click "Place Order". System: Save Order (implicit action).
5. Barista: Receive New Order Notification.
6. Barista: Start Preparing (updates status to “Preparing”).
7. Barista: Finish Preparation (updates status to “Ready”).
8. Customer: Receive "Ready" Notification (via tracking page).
9. Barista: Hand Over Coffee.
10. Barista: Mark as Completed.
11. End: Order completed.

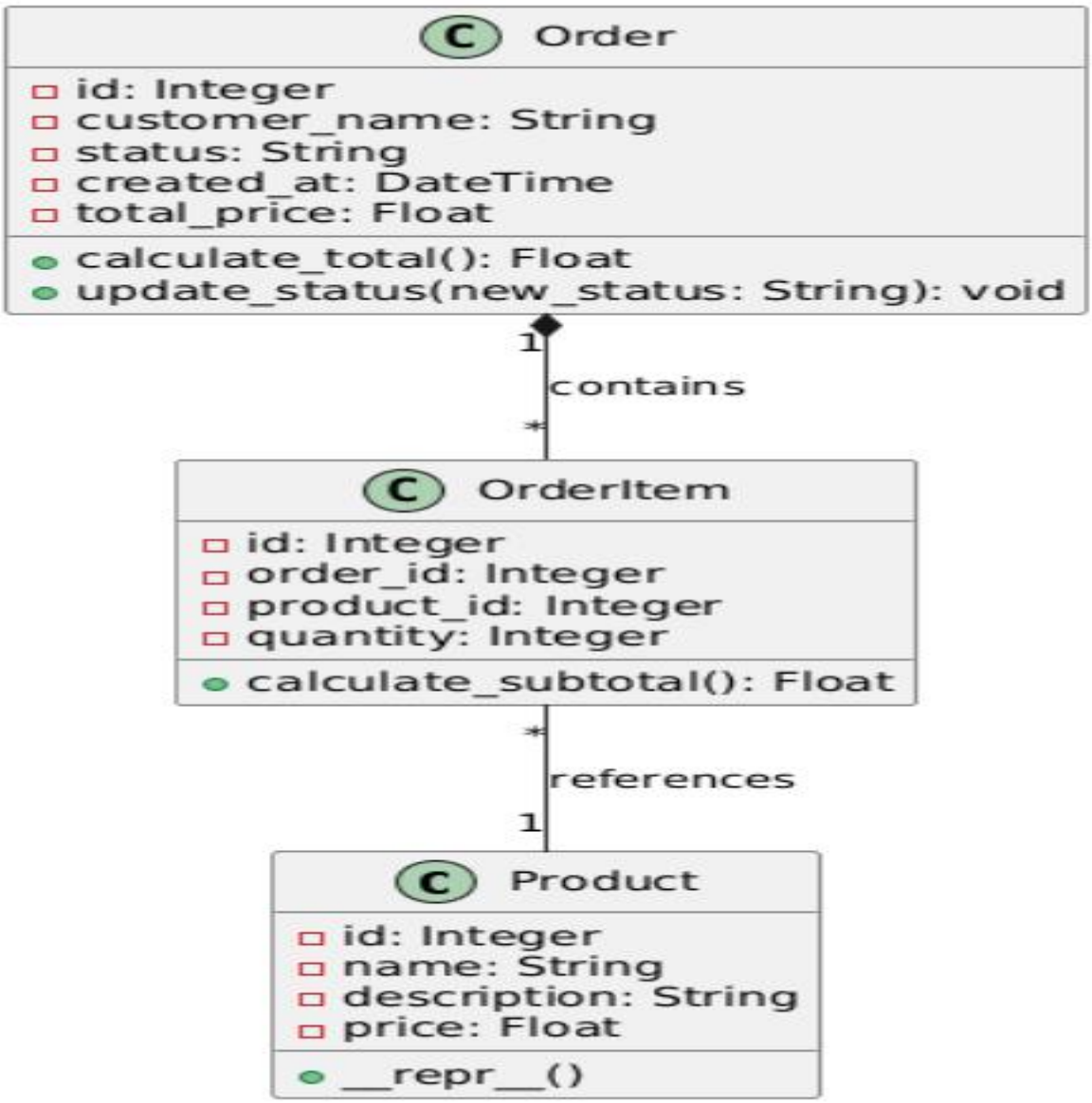

Customer
Barista
Browse Menu
Select Item(s)
View Cart
yes
Cart empty?
no
Browse Menu
Click "Place Order"
Order saved (system)
Receive New Order Notification
Start Preparing (status = "Preparing")
Finish Preparation (status = "Ready")
Receive "Ready" Notification
Hand Over Coffee
Mark as Completed (status = "Completed")

4 Class Diagram

Purpose: To show the static structure of the system, including domain classes, their attributes, methods, and relationships.

The main classes are:

1. Product: Attributes: id: Integer, name: String, description: String, price: Float Methods: __repr__()
2. OrderItem: Attributes: id: Integer, order_id: Integer (FK), product_id: Integer (FK), quantity: Integer Methods: calculate_subtotal()
3. Order: Attributes: id: Integer, customer_name: String (optional), status: String, created_at: DateTime, total_price: Float Methods: calculate_total(), update_status(new_status)
4. Relationships: Order 1 —— * OrderItem (composition) OrderItem * —— 1 Product (association)



5 State Machine Diagram

Purpose: To model the lifecycle of a single Order object, showing all possible states and the events that trigger state transitions.
The Order's lifecycle consists of four states:
1. Pending (initial state)
2. Preparing
3. Ready
4. Completed (final state)

Transitions:
Order Placed → moves from (none) to Pending.
Barista clicks "Start Preparing" → Pending → Preparing.
Barista clicks "Mark Ready" → Preparing → Ready.
Barista clicks "Complete" → Ready → Completed.
No other transitions are allowed (e.g., cannot go directly from Pending to Ready).